

Software Requirements Specification (SRS)

AI Interview Preparation Platform

Document Type: Requirements Report

Version: 1.0

Date: September 2026

Status: Approved

Table of Contents

1. Introduction
2. Overall Description
3. System Features
4. External Interface Requirements
5. Non-Functional Requirements
6. System Constraints
7. Data Flow Diagrams

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the AI Interview Preparation Platform. It is intended for use by the development team, project stakeholders, and QA engineers.

1.2 Scope

The platform is a full-stack web application that:

- Simulates AI-powered interview sessions for technical and HR interviews
- Provides personalized question generation using HuggingFace LLMs
- Analyzes user-uploaded resumes for ATS compatibility
- Tracks user progress with an analytics dashboard
- Allows scheduling of mock interview sessions

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification
JWT	JSON Web Token — stateless authentication mechanism
LLM	Large Language Model
DSA	Data Structures and Algorithms
HLD	High-Level Design
LLD	Low-Level Design
ATS	Applicant Tracking System
API	Application Programming Interface
SPA	Single Page Application
ODM	Object Document Mapper
DFD	Data Flow Diagram

1.4 References

- IEEE 830-1998 SRS Standard
- HuggingFace Inference API Docs
- MongoDB Documentation
- React 18 Documentation
- Project README, v1.1 (AI Interview Preparation Platform)

2. Overall Description

2.1 Product Perspective

The platform is a standalone web application composed of:

- A React SPA (client)
- A Node.js/Express REST API (server)
- A MongoDB database
- An external HuggingFace AI integration

[User Browser] <--> [React Frontend] <--> [Express API] <--> [MongoDB]
|
[HuggingFace AI API]

2.2 Product Functions Summary

Function	Description
User Registration & Login	JWT-based secure authentication
AI Question Generation	Topic-specific questions via HuggingFace LLM
Interview Simulation	Timed, real-time interview sessions
AI Answer Evaluation	Automated scoring and feedback
Resume Analysis	PDF parsing and ATS keyword checking
Question Bank	Searchable and filterable practice library
Progress Dashboard	Charts and analytics for preparation tracking
Mock Interview Scheduling	Calendar-based session scheduling

2.3 User Classes

Class	Description
Guest	Unauthenticated visitor; can view landing page only
Registered User	Authenticated user with full platform access
Administrator	Can manage question banks, users, and AI configuration

2.4 Operating Environment

- Server: Node.js 20.x on any Linux/Windows server
- Database: MongoDB 7.x (local or Atlas)
- Client: Modern web browser (Chrome, Firefox, Edge — last 2 versions)
- Network: HTTPS in production

2.5 Design and Implementation Constraints

- Observer Pattern is the primary design pattern for event-driven operations
- Layered architecture: Routes → Controllers → Services → Models
- JWT tokens are stateless; no server-side session storage
- All AI calls are made server-side to protect the API key
- Frontend state management uses the React Context API, scoped into Auth, Interview, and Progress contexts, per the client's context/ module
- Backend business logic and Observer Pattern implementations are isolated into dedicated services/ and observers/ modules to keep controllers thin

3. System Features

3.1 User Authentication Module

Description: Handles user registration, login, profile management, and session security.

Stimulus/Response:

- User submits registration form → System hashes password, stores user, returns JWT
- User submits login form → System validates credentials, returns JWT

Functional Requirements:

ID	Requirement
FR-01	The system shall allow users to register with a unique email and password
FR-02	The system shall validate email format and enforce minimum password length of 8 characters
FR-03	The system shall hash passwords with bcryptjs (salt rounds ≥ 10) before storage
FR-04	The system shall return a signed JWT (expires in 7 days) on successful login or registration
FR-05	The system shall expose a GET /auth/profile endpoint requiring valid JWT
FR-06	The system shall allow authenticated users to update name and profile details

3.2 AI Question Generation Module

Description: Dynamically generates interview questions using HuggingFace LLMs based on user-specified parameters.

Functional Requirements:

ID	Requirement
FR-07	The system shall accept topic, difficulty (easy/medium/hard), and role as generation parameters
FR-08	The system shall invoke the HuggingFace Inference API with a structured prompt
FR-09	The system shall parse and return structured question objects (question, hints, expected answer)
FR-10	The system shall cache repeated generation requests for the same parameters (optional optimization)
FR-11	The system shall handle HuggingFace API errors gracefully and return a fallback question

3.3 Interview Simulation Module

Description: Provides an end-to-end timed interview experience with AI-driven feedback.

Functional Requirements:

ID	Requirement
FR-12	The system shall create an InterviewSession record on session start
FR-13	Each session shall contain a configurable set of AI-generated or bank questions
FR-14	The system shall enforce a per-question time limit configurable by the user
FR-15	The system shall accept text-based answer submissions
FR-16	The system shall support voice-based answer input using browser Web Speech API
FR-17	On session completion, the system shall emit a session:completed event (Observer Pattern)
FR-18	All registered observers (ProgressObserver, ScoreObserver, FeedbackObserver, NotificationObserver) shall execute on completion
FR-19	The system shall return a complete post-session debrief including per-question scores and AI feedback

3.4 Resume Analysis Module

Description: Parses uploaded PDF resumes and provides AI-driven feedback.

Functional Requirements:

ID	Requirement
FR-20	The system shall accept PDF file uploads via multipart form-data (max 5MB)
FR-21	The system shall use pdf-parse to extract plain text from the PDF
FR-22	The system shall send extracted text to the AI for analysis
FR-23	The AI shall identify strengths, experience gaps, and missing ATS keywords
FR-24	The system shall store analysis results in the database linked to the user
FR-25	The system shall return historical resume analysis records on request

3.5 Question Bank Module

Description: A curated, searchable library of interview questions across categories such as DSA, System Design, HR, and Core CS.

Functional Requirements:

ID	Requirement
FR-26	The system shall store questions in MongoDB with fields: text, category, topic, difficulty, answer
FR-27	The system shall support filtering by category, topic, and difficulty via query parameters
FR-28	The system shall support keyword search in question text
FR-29	The system shall return paginated results (default page size: 20)

ID	Requirement
FR-30	Admins shall be able to add, edit, and delete questions via authenticated endpoints

3.6 Progress Tracking and Analytics Module

Description: Tracks user performance and surfaces insights via a dashboard.

Functional Requirements:

ID	Requirement
FR-31	The system shall update user progress automatically after each session via ProgressObserver
FR-32	The dashboard endpoint shall return: total sessions, average score, topic breakdown, weak areas
FR-33	The system shall track consecutive study days and return streak data
FR-34	The frontend shall render progress as line charts, radar charts, and scorecards

3.7 Mock Interview Scheduling Module

Description: Allows users to schedule future mock interview sessions.

Functional Requirements:

ID	Requirement
FR-35	The system shall allow users to schedule an interview for a future date/time
FR-36	The system shall store schedule records with user ID, scheduled time, and topic configuration
FR-37	The system shall return a list of upcoming scheduled sessions for a user

4. External Interface Requirements

4.1 User Interfaces

- SPA rendered by React 18 with React Router for client-side navigation
- Responsive design supporting viewports from 320px (mobile) to 1920px (desktop)
- Dark/light mode support

4.2 API Interfaces

Base URL: <http://localhost:5000/api>

Group	Base Path	Auth Required
Authentication	/auth	No (register/login)
Interview Sessions	/interview	Yes
Question Bank	/questions	Yes
Resume Analysis	/resume	Yes
Progress Analytics	/progress	Yes

All requests and responses use Content-Type: application/json. File uploads use multipart/form-data.

4.3 External AI Interface

- Provider: HuggingFace Inference API
- Model: mistralai/Mistral-7B-Instruct-v0.1 (configurable via HUGGINGFACE_MODEL_ID environment variable)
- Communication: HTTPS POST to <https://api-inference.huggingface.co/models/{model}>
- Auth: Bearer token via HUGGINGFACE_API_KEY environment variable

4.4 Database Interface

- Driver: Mongoose ODM connecting to MongoDB via MONGO_URI
- Collections: users, questions, interviewsessions, resumes, schedules

5. Non-Functional Requirements

5.1 Performance

- Standard API responses: ≤ 2 seconds under normal load
- AI-powered responses: ≤ 10 seconds
- Frontend initial load: ≤ 3 seconds on 4G connection

5.2 Security

- All passwords hashed with bcryptjs (salt rounds ≥ 10)
- JWT required for all protected endpoints; verified via authMiddleware.js
- File uploads: type validation (PDF only), size limit (5MB)
- Environment variables used for all secrets; no hardcoded credentials

5.3 Reliability

- Target uptime: 99.5%
- Graceful error handling on all API routes via errorHandler.js
- AI service failures return fallback responses, not unhandled exceptions

5.4 Maintainability

- Architecture: Routes $\rightarrow$ Controllers $\rightarrow$ Services $\rightarrow$ Models (strict separation)
- Observer Pattern for all post-session side effects — ProgressObserver, ScoreObserver, FeedbackObserver, and NotificationObserver — decoupled and testable
- Frontend state isolated into React Context modules (Auth, Interview, Progress) rather than prop-drilled or global state
- DRY utilities: apiResponse.js, validators.js
- Code linted with ESLint + Prettier

5.5 Scalability

- Stateless JWT authentication supports horizontal backend scaling
- MongoDB Atlas supports cloud-native horizontal scaling
- Frontend is a static SPA deployable to any CDN

6. System Constraints

Constraint	Detail
AI Dependency	Platform requires active HuggingFace API access
Database	MongoDB only; no relational DB support
File Format	Resume analysis supports PDF only
Browser	Voice input requires browser microphone permission grant
Environment	Node.js 18+ required for backend runtime

7. Data Flow Diagrams

This section presents the system's data flow using standard DFD notation: rounded rectangles for processes, open-ended rectangles for data stores, and labeled rectangles for external entities. The context diagram (Level 0) shows the platform as a single process; the Level 1 diagram decomposes it into its seven functional processes matching Section 3. A supporting sequence diagram illustrates the registration and protected-route request flow at the message level.

7.1 Context Diagram (Level 0)

The context diagram shows the AI Interview Preparation Platform as a single process boundary, with its four external entities: the User/Candidate, the Administrator, the HuggingFace Inference API, and the MongoDB database.

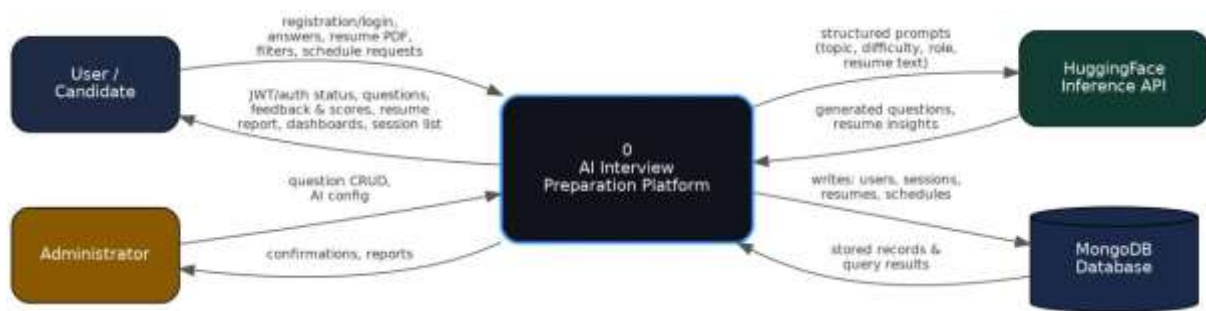


Figure 7.1 — Context-level (Level 0) Data Flow Diagram

7.2 Level 1 Data Flow Diagram

The Level 1 DFD decomposes the platform into its seven functional processes (1.0–7.0), each corresponding to a module in Section 3, along with the six data stores (D1–D6) and the external entities that produce or consume each flow. This diagram is rendered on a landscape page for legibility; refer to the following page.

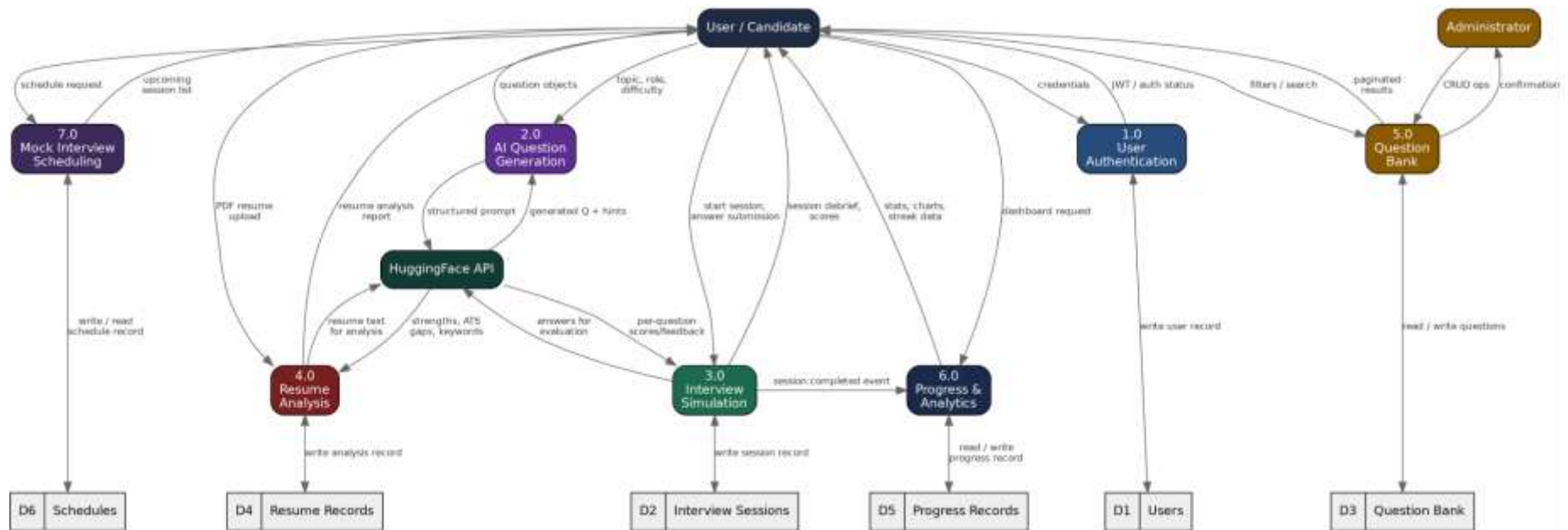


Figure 7.2 — Level 1 Data Flow Diagram (processes 1.0–7.0, data stores D1–D6)

7.3 Sequence Diagram — Registration & Protected Route Access

This sequence diagram traces a request/response flow at the message level for two related scenarios covered by FR-01 through FR-05: new-user registration, and a subsequent authenticated request to a protected route. It complements the DFDs above by showing ordering and the role of the Auth Middleware, which is folded into the single 'platform' process in the DFDs.

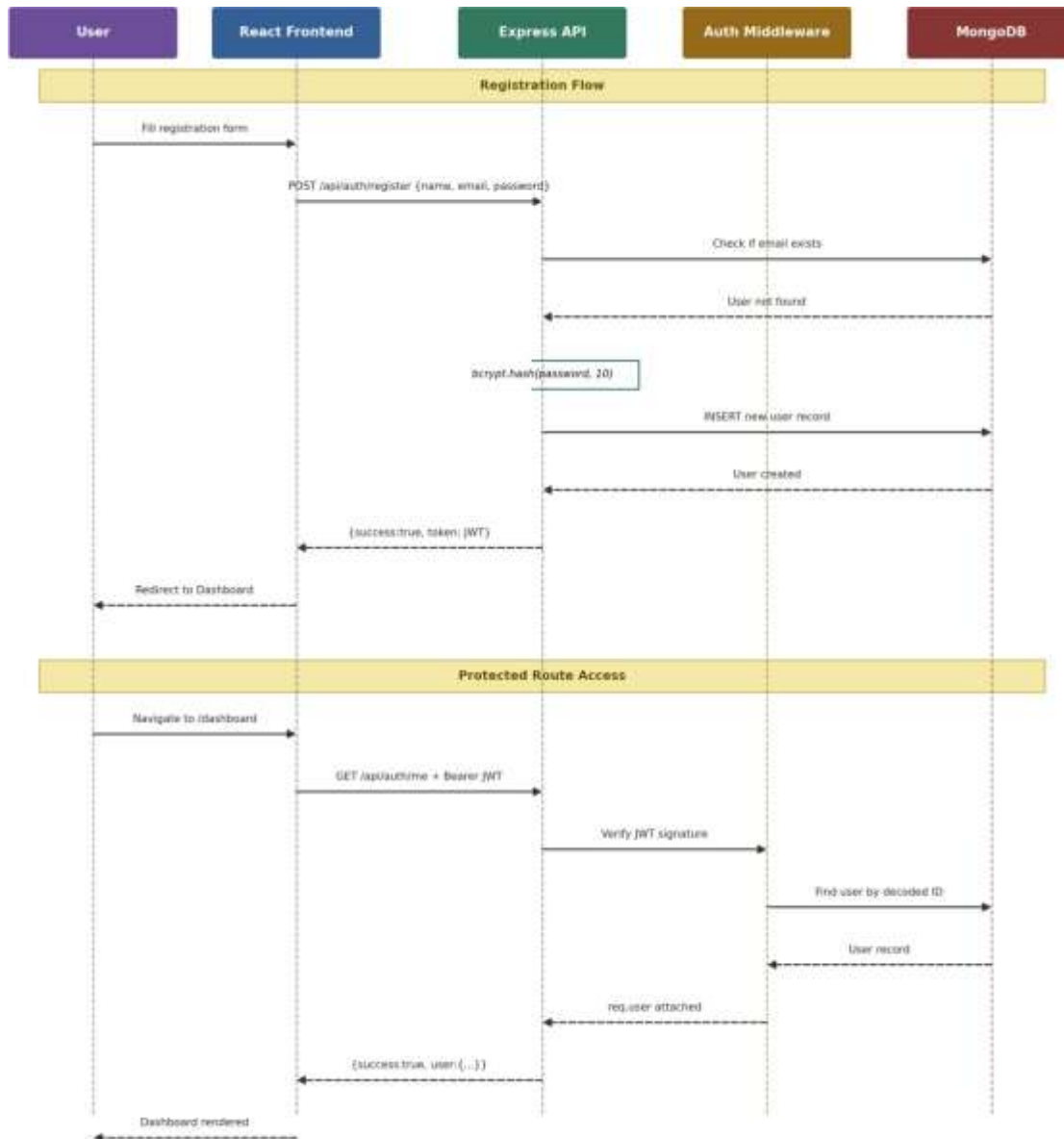


Figure 7.3 — Registration flow and protected-route access sequence

End of SRS Document